Lightweight Graph Partitioning Enhanced by Implicit Knowledge

Zhigang Wang, *Member, IEEE*, Gongtai Sun, Ning Wang, Lixin Gao, *Fellow, IEEE*, Chuanfei Xu, Yu Gu, Ge Yu, *Senior Member, IEEE*, and Zhihong Tian, *Senior Member, IEEE*

***Abstract*—Graph partitioning as a classic NP-complete problem, is the most fundamental procedure that needs to be performed before parallel computations. Partitioners can be divided into vertex- and edge-based approaches. Recently, both approaches are employing a streaming heuristic to find approximate solutions. It is lightweight in space and time complexities, but suffers from suboptimal partitioning quality, especially for directed graphs where the explicit knowledge provided for heuristic is limited. This paper thereby proposes new heuristics for not only vertex-based but also edge-based partitioning. They improve quality by additionally utilizing implicit knowledge, which is embedded in the local streaming view and the global graph view. Memory reduction techniques are presented to extract this knowledge with negligible space costs. That preserves the lightweight advantages of streaming partitioning. Besides, we study parallel acceleration and restreaming, to further boost the partitioning efficiency and quality. Extensive experiments validate that our proposals outperform the state-of-the-art competitors.**

***Index Terms*—Parallel graph processing, streaming graph partitioning, restreaming partitioning, directed graphs.**

Received 8 November 2024; revised 9 July 2025; accepted 23 August 2025. Date of publication 24 September 2025; date of current version 10 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62572139, Grant 62572108, Grant U2436208, and Grant 62372129, in part by Guangdong S&T Program under Grant 2024B0101010002, in part by Key-Area Research and Development Program of Guangdong Province under Grant 2023B1111050010, and in part by the Key Program of the Key Laboratory of Computing Power Network and Information Security under Grant 2024ZD016. Recommended for acceptance by M. Kandemir. *(Gongtai Sun is co-first author.) (Corresponding author: Zhigang Tian.)*

Zhigang Wang, Ning Wang, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Key Laboratory of Industrial Control System Security, and Huangpu Research School of Guangzhou University, Guangzhou, Guangdong 510006, China (e-mail: wangzhiganglab@gmail.com; wangninggzu@gmail.com; tianzhihong@gzhu.edu.cn).

Gongtai Sun is with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao, Shandong 266100, China (e-mail: sgt@stu.ouc.edu.cn).

Lixin Gao is with the Department of Electrical and Computer Engineering, University of Massachusetts Amherst, Amherst, MA 01003 USA (e-mail: lgao@ecs.umass.edu).

Chuanfei Xu is with the Central Research Institute, Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, Guangdong 518132, China (e-mail: xuchuanfei@gml.ac.cn).

Yu Gu and Ge Yu are with the School of Computer Science and Engineering, Northeastern University, Shenyang, Liaoning 110819, China (e-mail: guyu@mail.neu.edu.cn; yuge@mail.neu.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3612730

I. Introduction

LARGE graphs have billions of vertices and edges. Complex graph analysis computations require to be parallelized across distributed workers for scalability [1], [2]. However, before that, the input graph must be divided into different subgraphs/partitions for parallel computations.

A high **quality** partitioner should minimize the cross-partition dependency as the objective, subject to the roughly balanced workload distribution. Such requirements reduce communication costs and avoid the straggler issue in parallel computations. Besides, partitioning as a preprocessing operation, should be completed as quickly as possible (**efficiency**). Currently, there exist two kinds of partitioners, vertex partitioning (**VP**) and edge partitioning (**EP**). They respectively divide vertices and edges, where edges and vertices among partitions are cut and then generate communication costs [2], [3]. *VP* and *EP* offer different performance trade-offs in quality and efficiency. They are widely used in mainstream systems [1], [2], and hence are both studied in this paper.

Existing optimization efforts on *VP* and *EP* can be categorized into offline and online/streaming heuristic solutions [4]. The former regards *the whole graph* as a **global view**, and then performs multiple scans to continuously refine data assignments [3], [5], [6]. It provides high-quality partitions but suffers from expensive runtime and memory costs. Differently, a graph can also be ingested as a stream. This is motivated by the fact that the input graph should be loaded from disk before parallel analysis [1], [2]. Partitioning can be done during this loading phase without incurring additional data accesses. As vertices/edges arrive, we determine the location of *a new arrival* by *the distribution of already assigned data* (**local view**) [2], [7], [8], [9], [10]. Due to the fast one-pass data scan and the small local view, the lightweight streaming partitioner can easily scale to large graphs. However, the quality is suboptimal due to the lack of the global view.

We are aware that recently many pioneers have attempted to remedy these problems. They accelerate offline partitioners by parallel processing [11], [12], [13], [14], [15]. But the resulting variants still cannot work as efficiently as streaming competitors. They also improve the quality of streaming partitioners by combining existing offline and online works [16], [17], [18], [19]. The intuition behind these works is to compromise efficiency for quality, or vice versa. But by our tests, all of them cannot hit the "sweet spot" very well.

0018-9340 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

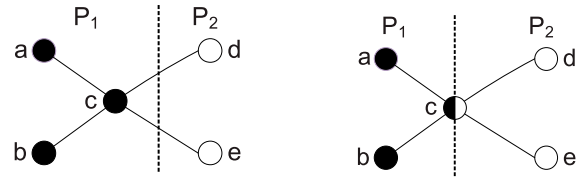
This paper stresses that there still exists huge space for quality improvement of a streaming method, with negligible compromise in its lightweight features. We aspire to this desirable goal by utilizing knowledge not only explicitly but also implicitly embedded in the local and global views.

We first focus on *VP*. We challenge the conventional wisdom that a streaming method can only extract explicit knowledge from the local view of a directed graph. Now both explicit out-neighbors and implicit in-neighbors are considered. That significantly reduces the number of cutting edges incurred by not only outgoing but also incoming edges. Moreover, we enrich knowledge by utilizing the topology locality, which is naturally implicitly embedded in the disk storage order of the whole input graph (global view).

We present lightweight techniques to extract implicit knowledge. We quantify local knowledge as how many times a vertex as an in-neighbor is expected to be assigned to a partition. This expectation counter can be incrementally updated during partitioning. We further group counters into shards and slide the update window across shards to reduce the required memory consumption. We identify common neighbors shared among multiple vertices as global topology locality knowledge. To preserve graph connectivity, vertices adjacent to these common neighbors are strategically assigned to the same partition, thereby minimizing edge cuts between them. This knowledge is efficiently extracted through an ID-based pre-assignment scheme that operates without requiring physical disk reads. Except for quality, we also batch streamed vertices to parallelize the process of making assignment decisions. A hash table is used to schedule concurrently assigned vertices, to mitigate the negative impact on quality.

Notably a graph can be reloaded repeatedly across multiple jobs, like multi-source traversals (e.g. Shortest Path) where each source vertex can issue a separate job. During the multiple loading phases, existing works have validated that the quality can be refined by again streaming partitions achieved in the previous pass [20], [21], [22]. However the cost-effectiveness is still an open problem. Existing restreaming solutions either select vertices to be restreamed statically (yielding marginal quality improvements) or employ global analysis (incurring substantial runtime overhead). Our proposed dynamic estimation mechanism effectively balances partitioning quality with computational efficiency. Besides, we cache restreamed vertices and mange data replacement with a novel Largest-Contiguous Used (LCU) policy, to mitigate the I/O bottleneck.

We further extend the utilization of implicit knowledge from *VP* to *EP*. Different from existing edge-centric partitioners, we group edges from an adjacency list into several fragments and then perform fragment-centric partitioning, so that implicit knowledge can normally work. Enhanced by the knowledge, our partitioner demonstrates comprehensive success compared against existing works. Moreover, many modern partitioners and systems simultaneously employ both vertex- and edge-based partitioning strategies to accommodate diverse graph datasets [23], [24], [25]. We emphasize that our two knowledge extraction principles are universally applicable to both partitioning paradigms. This dual compatibility significantly simplifies system design while maintaining flexibility.



(a) Vertex partitioning (b) Edge partitioning

Fig. 1. Two kinds of graph partitioning.

We state that in our early work the implicit knowledge has been used for quality improvement, but only for *VP* [26]. Now new contributions are presented, including a restreaming design for *VP* (Sec. V) and a generalization of our idea to *EP* (Sec. VI). We also make many engineering efforts in experiments, including rewriting our proposals in C++, running recently released partitioners as competitors, and using more graph datasets with different features.

II. Preliminaries

Given a directed graph $G = (V, E)$ with V as the set of vertices and E as the set of edges, we assume that vertices are consecutively numbered. For an edge $(v, u) \in E$, v as the source links to u as the target. The in- and out-neighbors of v are denoted by $N^{\text{in}}(v)$ and $N^{\text{out}}(v)$, respectively. Let $|\cdot|$ denote the size of a given set. The objective of graph partitioning is to divide G into K disjoint partitions $P_i = (V_i, E_i)$ with reduced connections among them, s.t. balanced workloads measured by the numbers of vertices $|V_i|$ or edges $|E_i|$.

The partitioning heuristics are divided into vertex- and edge-based variants. The former assigns vertices together with their adjacent edges where an edge (v, u) might be cut if v and u are placed into different partitions. The number of cutting edges clearly measures the connection between two partitions and hence dominates the communication cost in downstream graph processing jobs. The latter focuses on assigning edges, but an end-point vertex might be cut/replicated across partitions if edges with it as source or target have different assignment decisions. The communication cost is then dominated by the total number of replicas/cutting vertices. Fig. 1 distinguishes the two variants. In Fig. 1(a), c and d are assigned to P_1 and P_2, respectively, and hence (c, d) is cut. In Fig. 1(b), c is replicated in P_1 and P_2 because(a, c) and (c, d) are placed into the two partitions.

Graph processing systems, typically like vertex-based *Pregel* [1] and edge-based *PowerGraph* [2], run the corresponding partitioner as a built-in block before parallel computations. The partitioning runtime is thereby a portion of the total runtime and should also be reduced.

This paper focuses on vertex- and edge-based partitioning with high quality (reduced connection and balanced workload distribution among partitions) and efficiency (runtime). Below,

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

Secs III–V give our solutions for vertex-based partitioning. Sec VI extends our proposals to edge-based partitioning.

III. SPNL: Enhanced Streaming Partitioning

This section first introduces the classic existing heuristic as a basis, and then proposes our enhanced heuristics by considering in-out neighbors and topology locality.

A. LDG: The Basic Linear Deterministic Greedy Heuristic

LDG is a representative streaming partitioner [7]. As shown in Eq. (1), given an adjacency list with v as source, it tries to place it into the partition where a maximum number of v's adjacent neighbors have resided, while simultaneously satisfying the capacity constraint for workload balance. For our focused directed graphs, here only out-neighbors can be used to score candidate partitions and then select the best one with id pid. V_i^{pt} indicates till now, at the time instance t, what vertices have already arrived and placed into the i-th partition P_i. $w^t(i,v)$ is the real-time remaining workload capacity, which works as a penalty function. We measure it by $1-\frac{|P_i^t|}{C}$, where the pre-set capacity C and the workload $|P_i^t|$ can be respectively $\frac{|V|}{K}$ and $|V_i^{pt}|$ for balanced distribution of vertices; and $\frac{|E|}{K}$ and $|E_i^{pt}|$ for edges.

$$\text{pid} = \arg\max_{i\in[1,K]} \left\{ w^t(i,v) \cdot \left|V_i^{pt} \cap N^{\text{out}}(v)\right| \right\} \tag{1}$$

B. A Heuristic Enhanced by Runtime In-Out Neighbors

LDG runs very quickly since data is scanned only once. However, it cannot reduce the number of cutting edges very well. The reasons are twofold. The first is that if v's out-neighbors have been roughly evenly placed among partitions, the score is then mainly dominated by the remaining capacity. In other words, it assigns v without any optimization about cutting edge reduction. The second is that at the initial streaming stage, all partitions are roughly empty. The nearly equal score values enforce *LDG* to blindly randomly assign v.

In fact, the two reasons arise from the limited knowledge extracted from the **Local View** in data stream, which consists of the currently arrived adjacency list and the distribution of already placed vertices. We thereby propose a new Streaming Partitioner based on both in- and out-Neighbors (**SPN**), so as to enrich knowledge. The idea behind it is that a vertex v not only sends messages along outgoing edges to its target out-neighbors, but also receives messages along incoming edges from in-neighbors. As demonstrated in the top of Fig. 2, assigning v to the partition, to which a maximal number of its out-neighbors have been placed, can reduce the number of cutting (outgoing) edges linking from v. Meanwhile, assigning it to the partition with the maximal number of in-neighbors, can also achieve the same goal, but now the reduction is contributed by incoming edges linking to v. Motivated by this insight, we modify Eq. (1) by additionally considering the impact of in-neighbors, i.e., $\left|V_i^{pt} \cap N^{\text{in}}(v)\right|$.

Note that in-neighbors are not available in the adjacency list of a directed graph. Reconstructing the in-neighbor list as a preprocessing phase can solve this problem but introduces additional I/O overhead when loading the input graph. Our alternative is to partially infer this in-neighbor information from local view. At time t, we maintain a lookup table Γ_i^t of size $|V|$, to count how many times an arbitrary vertex $v \in V$ exists as an out-neighbor of vertices already assigned to P_i. Assume that u is being placed into P_i. We update Γ_i^t by traversing all of its out-neighbors. Specifically, for an edge $(u,v) \in E$, $\Gamma_i^t(v)$, the counter associated with v, is increased by one. This counter stores how many vertices in P_i, like u, expect v to be assigned to the same partition. Accordingly, when processing v later, we can estimate $|V_i^{pt} \cap N^{\text{in}}(v)|$ by $\Gamma_i^t(v)$. Note that some vertices linking to v perhaps have not been streamed at t. Such runtime **expectation estimation** might be smaller than the true value, but the impact on the score distribution is big enough for making smart placement decision. The heuristic now is re-written as Eq. (2).

$$\text{pid} = \arg\max_{\text{i}\in[1,\ \text{K}]} \left\{ w^t(i,v) \cdot \left(\left|V_i^{pt} \cap N^{\text{out}}(v)\right| + \Gamma_i^t(v) \right) \right\} \tag{2}$$

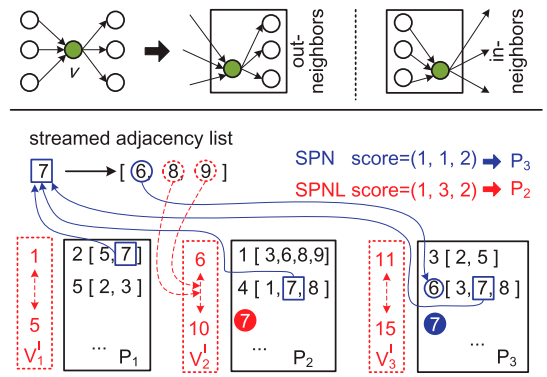


Fig. 2. Illustration of *SPN* (solid blue line) and *SPNL* (dotted red line).

Fig. 2 demonstrates how *SPN* works with $K=3$. Vertices numbered from 1 to 6 have been serially streamed and placed by $V_1=\{2,5\}$, $V_2=\{1,4\}$, and $V_3=\{3,6\}$. For the currently arrived vertex 7 with $N^{\text{out}}(7)=\{6,8,9\}$, by Eq. (2), we know that the out-neighbors-based score is (0,0,1) and the in-neighbors based expectation estimation score is (1,1,1). Vertex 7 is finally assigned to P_3 due to the combined score (1,1,2).

C. A Heuristic Enhanced by Topology Locality

Although in-neighbor expectation enriches knowledge, at the initial streaming stage, the effectiveness is still marginal. This is because only a few neighbors are streamed. As shown in Fig. 2, for vertex 7, the out-neighbors 8 and 9 have not been assigned and hence cannot provide any useful knowledge.

In this section, we design a two-step variant of *SPN* with a very lightweight preprocessing step where all vertices (**Global View**) are pre-assigned. Then at the second actual assignment step, such pre-assignments V_i^{lt} as prior-knowledge can be used to improve partitioning quality, as shown in Eq. (3). Here we separately compute the two intersection sets w.r.t. V_i^{pt} and

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

V_i^{lt}, and weight them with a decay factor η. η_i^t gradually decreases to strengthen the impact of already assigned data, when more vertices are streamed. This paper typically sets $\eta_i^t = \max\{0, \frac{|V_i^{lt}|-|V_i^{pt}|}{|V_i^{lt}|}\}$, and more interesting yet effective settings will be explored as future work. Note that V_i^{lt} also dynamically changes. $v \in V_i^{lt}$ will be immediately removed once it has been actually assigned.

$$\text{pid} = \arg\max_{\text{i}\in[1,\ \text{K}]} \Bigg\{ w^t(i,v) \cdot \Big(\Gamma_i^t(v) + \big((1-\eta_i^t)|(V_i^{pt}) \cap N^{\text{out}}(v)| + \eta_i^t|(V_i^{lt}) \cap N^{\text{out}}(v)|\big)\Big)\Bigg\} \quad (3)$$

The pre-assignment policy should (1) capture the topology locality for better partitioning quality, i.e., $v \in V_i^{lt}$ will be added into V_i in high probability; and (2) be lightweight with negligible memory consumption and runtime latency for better scalability. Notably graphs are crawled from real-world scenarios and then stored onto disk. Vertices closely crawled usually have the similar feature. For example, fans loving classical music can follow many common singers in this field. These singers as common neighbors exist in the adjacency lists of fans. Assigning fans and singers into the same partition can avoid cutting edges. This **common-neighbor-based locality knowledge** is naturally embedded into the disk storage order of crawled adjacency lists[1]. We thereby employ a *Range* method as the pre-assignment policy to preserve this raw locality, where vertices are consecutively distributed across partitions with the same disk storage order. Since vertices are continuously numbered, the pre-assignment route table just consists of the minimal and the maximal boundary vertex ids for each partition, leading to only $2K$ space consumption. Further, by evenly splitting data, these boundary ids can be quickly inferred without a physical data scan. We call this *SPN* variant enhanced by locality as **SPNL**.

Fig. 2 demonstrates *SPNL*. The total 15 vertices are evenly pre-assigned across 3 partitions. When vertex 7 arrives, we compute the score contributed by not only actually placed in-neighbor 2 and out-neighbor 6, but also pre-assigned out-neighbors 8 and 9 falling into V_2^l. The final score is (0,3,2)[2] and hence 7 is placed in P_2. After that, 7 is removed from V_2^l since it has been actually assigned.

IV. Lightweight Streaming Optimizations

This section introduces our optimizations on memory consumption and partitioning efficiency. They make *SPN* and *SPNL* work in a lightweight and scalable manner.

A. Memory Reduction With Sliding Window

SPN and *SPNL* output high-quality partitions but at expense of memory. Let $\max d$ be the maximal out-degree in G. In *LDG*, we need to cache an adjacency list of size $\max d$ at most, a K-dimensional score vector, and a route table about the placements of all $|V|$ vertices. The space complexity is $O(|V| + K + \max d)$. *SPN* additionally counts the expectation on all vertices for each partition. The consumption significantly increases to $O((K+1)|V| + K + \max d)$. *SPNL* further caches the pre-assignment route table, but the lightweight consumption of size $2K$ makes it still have the same space complexity with *SPN*. Clearly, expectation estimation with $O(K|V|)$ limits the memory scalability of our two partitioners.

In fact, in a streaming partitioner, actually assigned vertices will not be moved again. Then their expectation values are useless. That motivates us to release their occupied memory units. For each partition P_i, we can evenly divide all counters in Γ_i into X shards, S_{i1}, S_{i2}, ..., and S_{iX}. Since vertices are consecutively numbered and serially streamed, we safely abandon a shard S_{ij} once vertices within it have been actually assigned. The memory consumption thereby gradually decreases. However, at the very beginning, all X shards should be preserved and the peak consumption is still $O(K|V|)$.

Another observation is that benefiting from topology locality, the currently arrived vertex usually has edges linking to neighbors numbered roughly around it. This motivates us to slide a focus window from one shard to another along with the streaming operation, where only vertices in the window update expectation values. Then different shards can share memory spaces. The peak consumption is reduced to $O(\frac{K|V|}{X})$.

Clearly, a large X can significantly reduce the memory cost, but some useful counts might be missed and then impair the quality. Suppose the currently arrived vertex v falls into S_{ik} related to the k-th shard of the i-th partition. Given an out-neighbor $u \in N^{\text{out}}(v)$, it might belong to the (k+Δ)-th shard which will be focused on in future, that yields counting loss. The loss is especially large when v is the boundary vertex of S_{ik}, because its out-neighbors belong to the next shard $S_{i(k+1)}$ in high probability. We thereby employ a fine-grained sliding manner. The sliding unit is a vertex, rather than a shard. When a new vertex arrives, we remove the oldest one and extend the boundary to the next vertex. Under this design, in Sec. VII-B, we empirically give a policy to determine a reasonable X.

Fig. 3 gives an example with $X = K = 2$ and $|V| = 4$. Vertices 1, 2, and 3 are serially respectively assigned into P_1 and P_2. The shard size is $\left\lceil \frac{|V|}{X} \right\rceil = 2$. At the beginning, vertices 1 and 2 are covered by the window. Then $\Gamma_1(2)$ and $\Gamma_2(1)$ both increase by 1. However, $\Gamma_1(4)$ and $\Gamma_2(3)$ do not since 3 and 4 are excluded from the window, even though there exist edges $(1,4)$ and $(2,3)$. When the new vertex 3 arrives, the window slides to the next vertex. Now $\Gamma_1(1)$ is removed and $\Gamma_1(4)$ has not been covered, neither of which will be updated. The missing updates on Γ except $\Gamma_1(1)$ impair the quality.

B. Efficiency Improvement by Parallel Partitioning

Although the streaming partitioner has native efficiency advantages due to the single-pass data scan, the runtime scalability is still poor when processing large graphs. The main workload of streaming partitioning consists of loading data from the input disk file, and computing partition-specific scores for

[1]Please refer to Table I in Ref. [27].

[2]For better understanding, here we remove the weight parameter η.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

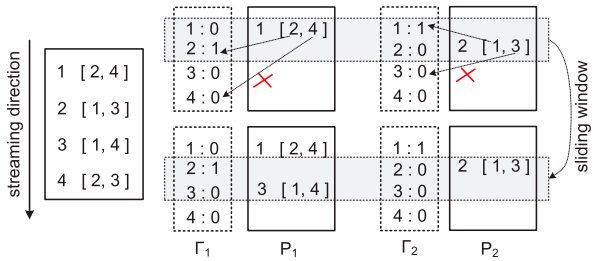


Fig. 3. Memory optimization by sliding window.

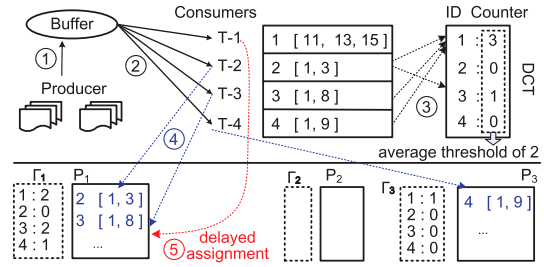


Fig. 4. Dependency detection optimization in parallel computations.

each vertex. The I/O bottleneck can be alleviated through high-performance storage solutions (e.g., SSDs). Our work primarily focuses on parallelizing scoring operations.

Our data-parallel implementation constructs a producer-consumer queue to buffer streamed data. Many threads concurrently take adjacency lists from the buffer and then compute scores. However, perhaps these vertices are also adjacent to each other. The heuristic information from their assignments will be ignored, if we assign them at the same time.

To alleviate the resulting quality degradation, we enforce these concurrent vertices to actively detect the adjacency dependency. The assignment of a vertex with many dependencies can be temporarily delayed so that it can fully utilize the heuristic knowledge from its in-out neighbors. However, this detection can be expensive, if, for example, we compute the intersection set of neighbors for any pair of buffered vertices.

Here we propose a hash-based Dependency-Counting-Table (*DCT*) that enables efficient dependency detection by maintaining vertex IDs paired with dependency counters. As shown in Fig. 4, the process begins with loading adjacency lists into the buffer (step ①), which are then fetched by multiple threads (step ②). Each thread traverses the edges of its assigned adjacency list to score candidate partitions while simultaneously updating the corresponding counters (step ③). A vertex increases its counter if it exists as either in-neighbor or out-neighbor in the concurrently fetched lists. Notably ID lookup achieves constant time complexity $O(1)$, while counter updates require cross-thread synchronization through locking mechanisms. Once all vertices complete their scoring operations, the dependency counters become available. Vertices compare their counters against an automatically calculated delay threshold (the average of non-zero counters) to determine whether to proceed with immediate assignment (step ④) or delay for re-scoring with updated partition statistics (Γ_i, V_i^{lt} and V_i^{pt}, step ⑤). In cases where all vertices have uniform counters, we perform assignments in a random order to prevent potential deadlocks while ensuring progress.

Consider the example in Fig. 4, where four threads (T_1-T_4) engage in parallel vertex assignments. Threads T_2-T_4 immediately process vertices 2-4, as their respective dependency counters (all below the average threshold of 2) permit concurrent assignment. Meanwhile, T_1 subsequently assigns vertex 1. It needs to recompute scores for partitions P_1 and P_3 to account for their updated statistics since they receive new data from the earlier parallel assignments.

V. ReSPNL: Fast Restreaming Partitioning

Now we present *ReSPNL*, the Restreaming variant of *SPNL* for better quality at expense of streaming data multiple passes.

A. Problem Analysis

Although *SPN* enhances partitioning quality through in-neighbor analysis, its effectiveness is still limited in certain scenarios. For instance, as shown in Fig. 2, the first vertex in the streaming sequence ($1 \rightarrow [3, 6, 8, 9]$) must be assigned randomly (to P_2 in this case) due to the lack of prior placement information. This arbitrary assignment subsequently causes edge cuts for edges $(1, 3)$ and $(1, 6)$ when vertices 3 and 6 are allocated to P_3. Moreover, the pre-assignment strategy employed in *SPNL* may introduce inaccuracies that misguide placement decisions. In the same example, vertex 7 is assigned to P_2 based on the pre-assignment of vertices 8 and 9 to P_2 (as indicated by V_2^l). However, if 8 and 9 are ultimately placed in P_3, $(7, 8)$ and $(7, 9)$ become additional cutting edges.

On the other hand, streaming partitioning is executed along with graph loading in graph processing systems. These systems frequently require reloading the same input data across multiple jobs—particularly for multi-source traversal algorithms like Shortest Path and Personalized PageRank computations, where each source vertex initiates a separate job. These repeated streaming passes (commonly termed restreaming) present opportunities to correct misplacements mentioned above to further enhance quality. The partitioning outcomes from the previous streaming pass can serve as more accurate pre-assignments. In *SPNL*, such pre-assignments can also compensate for the lack of initial placement knowledge for vertices streamed at the very beginning of a pass.

Fig. 5 illustrates the iterative restreaming framework. In each iteration, the partition assignments from the previous pass (step ①) serve as pre-assignments for the subsequent pass (step ②), which then guides *SPNL*'s score calculation (step ③). For instance, when vertex 1 is restreamed, it receives a score vector of (0, 0, 4) because all its out-neighbors reside in P_3, prompting its migration from P_2 to P_3 to eliminate four cutting edges. Meanwhile, the expectation counter table Γ is updated conventionally (step ④), with Γ_3 values for vertices 3, 6, 8, and 9 being set to 1. These counters serve as knowledge from these vertices' in-neighbor (vertex 1). For example, when 3 is restreamed, $\Gamma_3(3)$ contributes to the third element in the score vector (2, 0, 1).

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

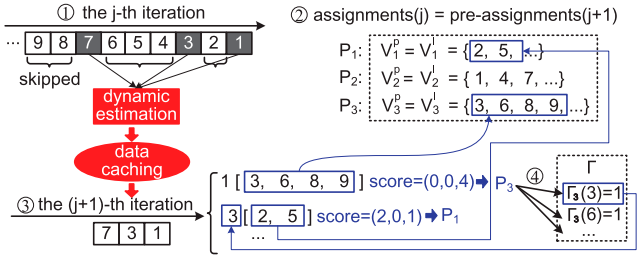


Fig. 5. The fast restreaming partitioning framework.

Through our analysis of the restreaming process, we identify two fundamental challenges. One is the selection criterion for determining which vertices should be restreamed. The other is the elimination of redundant data accesses when streaming vertices across iterations (e.g., vertices 8 and 9 are loaded but not re-assigned). To address these challenges systematically, we propose a dynamic estimation approach (Sec. V-B) for intelligent vertex selection, and a caching mechanism (Sec. V-C) for I/O-efficient data access.

B. Restreaming Heuristics With Dynamic Estimation

Because the data assignments continuously evolve across iterations, we should dynamically estimate the benefit of restreaming a specific vertex. We achieve this goal by measuring the degree of suboptimal assignments. We list three factors. The first is the low in- and out-degrees, respectively leading to the small expectation estimation $\Gamma_i^t(v)$ and out-neighbor set $N^{\text{out}}(v)$. The second is the early streamed location where few lists are assigned and hence V_i^{pt} is small. As shown in Eqs. (2) and (3), the two factors yield a weak relationship between the streamed list $\text{adj}(v)$ and a candidate partition P_i. The cutting edge optimization is then less significantly considered. Thirdly, if neighbors of v are evenly distributed among two or more partitions, it is difficult to distinguish partitions in terms of reducing the cutting edges.

To quantify the relative degree size, we normalize v's degree $(|N^{\text{in}}(v)| + |N^{\text{out}}(v)|)$ by the maximal degree of G, $\max d$. Further, at the j-th iteration, assume that v is the $L^j(v)$-th vertex in the set of streamed vertices V^j. $\frac{L^j(v)}{|V^j|}$ measures how early v is streamed. For the third, we need to compute the cutting-edge-reduction difference between the best target partition P_{bst} and the second best P_{sec}. Let $\Phi^j(v, \text{pid})$ indicate this reduction estimation $\left(|V_{\text{pid}}^{pt} \cap N^{\text{out}}(v)| + \Gamma_{\text{pid}}^t(v)\right)$ if v is assigned to P_{pid}. We also normalize the difference value $\Phi^j(v, \text{bst}) - \Phi^j(v, \text{sec})$ by $\Phi^j(v, \text{bst})$. Eq. (4) combines the three normalized sub-metrics through summation, since each corresponding factor contributes equally to the increase in cutting-edge count. When processing a vertex v at the j-th pass, it is put into V^{j+1} if its metric $\Psi^{j+1}(v)$ is lower than or at most equal to a threshold θ^j. Notably calculating $\Phi^j(v, \text{pid})$ in the last sub-metric requires intersection operations across all candidate partitions. However, this compute-intensive operation has already been completed during normal partitioning as shown in Eq. (2) and Eq. (3). Consequently, we can reuse these results without incurring additional runtime cost. This enables efficient estimation to be performed concurrently with the ongoing streaming partitioning process.

$$\Psi^{j+1}(v) = \frac{|N^{\text{in}}(v)| + |N^{\text{out}}(v)|}{\max d} + \frac{L^j(v)}{|V^j|} + \frac{\Phi^j(v, \text{bst}) - \Phi^j(v, \text{sec})}{\Phi^j(v, \text{bst})} \tag{4}$$

Notably it has been validated that in the streaming sequence prioritizing vertices by degrees can further enhance quality [22]. However, adjusting the streaming order one-by-one causes sorting costs. Our optimization is to divide vertices into several buckets by degrees, and then run a coarse-grained bucket-centric sorting operation.

Last but not least, benefiting from the continuous refinement, the restreaming gain gradually decreases. It is desirable to terminate the iterative passes in time. Directly setting the upper bound of passes is a challenging task, because an over-small value might prevent vertices from further assignment optimization. Our solution is to gradually decrease θ^j to reduce the number of selected vertices, and then naturally terminate passes when $V^* = \emptyset$ happens. Such a friendly termination provides enough re-assignment opportunities. More specifically, we decrease θ in a recursive damping manner, i.e., $\theta^{j+1} = \theta^j \cdot \frac{|\widetilde{V^j}|}{|V^j|}$. $|\widetilde{V^j}|$ indicates how many vertices in V^j indeed change their partition assignments. A small factor $\frac{|\widetilde{V^j}|}{|V^j|}$. $|\widetilde{V^j}|$ indicates that the selection is less effective. θ^{j+1} thereby decreases with a big step size. As a result, only vertices with large benefits can satisfy such a strict criterion.

As shown in Fig. 5, during the j-th iteration, many vertices are streamed but only 1, 3, and 7 are selected to be processed again in the (j+1)-th iteration. Note that if a vertex v (e.g., 8) is skipped in the (j+1)-th iteration, its estimation metric $\Psi^{j+2}(v)$ becomes unavailable. As a result, v is excluded from all subsequent iterations. That potentially limits its opportunity for re-assignment optimization. However, benefiting from the dynamic θ, this design incurs only negligible quality loss while achieving significant runtime savings.

C. I/O-Efficient Caching Mechanism for Restreaming Data

SPNL is I/O-efficient due to the one-pass sequential read on the disk-resident input graph. However, in *ReSPNL*, repeatedly loading the graph generates a lot of reads but many of them are redundant, because only partial vertices are selected for restreaming (e.g., vertices 1, 3, and 7 in Fig. 5).

A straightforward solution is to cache vertices across iterations. By Sec. V-B, our *ReSPNL* has a natural advantage. In the j-th iteration, we have already known whether or not a specific vertex will be processed again in the next iteration. Motivated by this, we can accurately cache data on demand, with a zero cache miss ratio.

Note that the number of selected vertices can be large. The required buffer size might exceed the capacity of available memory. As a result, I/O operations might still exist. Because vertices individually make their restreaming decisions, the locality of their distribution among disk page files is poor. For

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

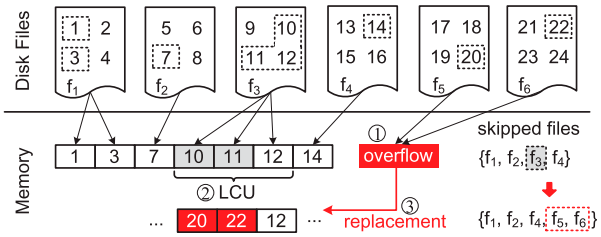


Fig. 6. Data replacement under LCU.

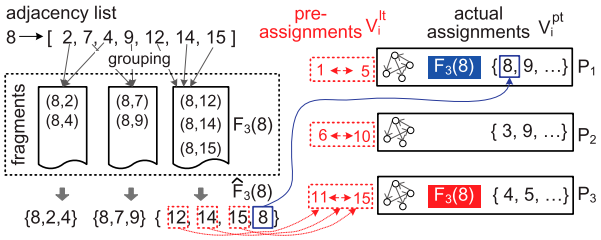


Fig. 7. Fragment-centric edge partitioning.

better I/O-efficiency, we design a Largest-Contiguous Used (LCU) policy to manage data replacement, so as to skip page files as many as possible. More specifically, LCU prefers to cache vertices non-consecutively distributed across page files. Then the number of pages covered by cached vertices increases, and these pages can be completely skipped.

Fig. 6 illustrates the LCU replacement policy in our caching design. Assume that vertices are stored across six disk page files (f_1 through f_6). Among them, the dotted rectangles highlight vertices are selected for the next restreaming iteration. We attempt to cache all selected vertices but encounter a buffer overflow when loading vertex 20 due to the capacity constraint (step ①). We identify that vertices from f_3 occupy the largest contiguous memory block (step ②). Under LCU, we replace vertices 10 and 11 (from f_3) with vertices 20 and 22 (from f_5 and f_6 respectively) in step ③. Now although f_3 requires reloading, we eliminate the need to access f_5 and f_6 in subsequent operations, improving the overall I/O efficiency.

VI. Streaming Partitioning: From Vertex to Edge

Secs. III-B and III-C introduce two knowledge enhancing principles for vertex-based partitioning. We now show their generic applicability by extending them to edge-based partitioning. We design a novel fragment-centric partitioner, to cope with new challenges arising from cutting vertices.

A. Overview of Fragment-Centric Streaming Partitioning

Recall that *EP* always assigns an edge to a partition where one or two of its end-points have already been cut/replicated. The core idea is to prevent end-points of newly streamed edges from becoming new replicas. Clearly, the current *EP* heuristic only cares about whether or not an existing replica can be shared, instead of how many times it is shared. Thus, the counter-based statistics, i.e., the expectation estimation counter $\Gamma_i^t(v)$ in Sec. III-B, cannot work. On the other hand, Sec. III-C shows that the effectiveness of topology locality, is directly proportional to the number of common neighbors that we can see across adjacency lists. However, an *EP* partitioner segments edges in a list and then streams them one by one. That means only one neighbor is seen in the local view, which heavily limits the effectiveness. A simple solution is to package all edges in a list, and then assign them as a whole, resembling *VP*. However, these edges usually have very different favorite partitions. This coarse-grained packaging method limits the flexibility of edge assignment, and hence, many vertices might be cut. That goes against the goal of *EP*.

Different from the traditional edge-centric partitioning and the *VP*-like list-centric variant, we propose a fragment-centric compromise. Given an adjacency list with source vertex v, its outgoing edges are grouped into several fragments, each of which $F_j(v)$ as a whole is assigned. Note that edges within $F_j(v)$ share the common source v, which avoids generating new replicas of v. By contrast, the other end-points of these edges, i.e., targets, still might be cut and then become new replicas. To reduce the number of these replicas, it is important to reasonably group edges. Sec. VI-B shows that topology locality as the global implicit knowledge can help to generate high-quality fragments. Sec. VI-C shows that assignment expectation as the local implicit knowledge can weight the importance of a fragment, to further enhance partitioning quality.

The resulting heuristic is mathematically shown in Eq. (5). $\widehat{F}_j(v)$ indicates the set of all vertices involved in edges in $F_j(v)$. V_i^{pt} stands for the set of vertices actually replicated in the i-th partition P_i at time t, while V_i^{lt} is the Range-based pre-assignment results. *part-1* and *part-2* in Eq. (5) respectively indicate the implicit knowledge of topology locality (pre-assignments) and expectation estimation.

$$\text{pid} = \arg\max_{i\in[1,K]} \Bigg\{ w^t(i,v)\cdot\Big(|\widehat{F}_j(v)\cap V_i^{pt}| + \underbrace{|\widehat{F}_j(v)\cap V_i^{lt}|}_{part-1} + \overbrace{\sum_{u\in\widehat{F}_j(v)} \oplus\|\Gamma_i^t(u)\|}^{part-2}\Big)\Bigg\} \tag{5}$$

Fig. 7 illustrates an example in which the adjacency list originating from source vertex 8 is partitioned into three fragments. Consider the third fragment, $F_3(8)$, its associated vertex set, $\widehat{F}_3(8)$, comprises four vertices connected by three edges within $F_3(8)$. By examining the relationship between $\widehat{F}_3(8)$ and the pre-assigned or actual replicas (V_i^{lt}/V_i^{pt}), edges within $F_3(8)$ are put into a certain partition.

B. Generating Fragments by Topology Locality

Intuitively, a fragment $F_j(v)$ should be assigned into the partition that has a maximum number of shared end-points/replicas. However, when streaming $F_j(v)$ at time t, we only know the distribution of partial replicas, i.e., V_i^{pt}. As a result, we cannot accurately score candidate partitions.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

Our compensation is to generate high-quality fragments to reduce the number of replicas. In theory, some edges in an adjacency list should be grouped into a fragment, if all or most of their targets also exist in other lists. Then these fragments from different lists can be assigned to the same partition to share these target replicas. However, such knowledge is not available in streaming partitioning. Besides, given a list with source v, we should enumerate all possible grouping combinations to select the best one. The computation complexity of enumeration is up to $O(\sum_{i=1}^{|N^{\text{out}}|} \text{Str}(|N^{\text{out}}|, i))$[3]. It is prohibitively expensive.

Fortunately, topology locality (see Sec. III-C) can provide the necessary information. By *Range* partitioning, the output pre-assignment route table V_i^{lt} tells us what vertices have the similar feature and are linked to together by other vertices. Edges are grouped into a fragment $F_j(v)$ if their targets fall into the same range specified by V_i^{lt}. As shown in Fig. 7, without pre-assignment, $F_3(8)$ would be assigned into P_1 because $\widehat{F}_3(8) \cap V_1^{pt} = \{8\}$ meaning vertex 8 does not introduce a new replica. However, $\widehat{F}_3(8) \cap V_3^{lt} = \{12, 14, 15\}$ suggests that subsequent fragments from other lists might share these same neighbors. If all such fragments are placed in P_3, their neighbor replicas can be shared. Accordingly, *part-1* in Eq. (5) employs $|\widehat{F}_j(v) \cap V_i^{lt}|$ to quantify the fragment quality to favorite partition P_i. Note that the grouping computation complexity now is reduced to $O(|N^{\text{out}}(v)|)$ because all fragments can be generated in a single traversal of edges.

C. Heuristics Enhanced by Replication Expectation

In Eq. (5), $|\widehat{F}_j(v) \cap V_i^{pt}|$ and $|\widehat{F}_j(v) \cap V_i^{lt}|$ respectively stand for impacts of building fragments (pre-assigned data) and sharing replications (actually assigned data). They cooperatively contribute to scoring a partition. In the early streaming stage, V_i^{pt} is small because few fragments are assigned. By contrast, V_i^{lt} plays an important role. As partitioning proceeds, $\widehat{F}_j(v) \cap V_i^{pt}$ becomes increasingly important. We consider such a change using the replication expectation as a tuning weight, i.e., *part-2* in Eq. (5). For any $u \in V_i^{pt}$, its expectation counter increases if it is the source or target of any edge in an already assigned fragment. This counter indicates how many times a replica has been shared in P_i. It is equal to $\Gamma_i^t(u)$ used in *VP*. $\Gamma_i^t(u)$ is direct proportional to $|V_i^{pt}|$, that can gradually strengthen the impact of actually assigned data.

Note that there exist many vertices in $\widehat{F}(v)$. When assigning the fragment $F_j(v)$, we need to carefully aggregate the impact of the corresponding expectations. Now we give different implementations for the abstract aggregation $\oplus$.

Accumulating Expectations (AE). A naive implementation is to sum up expectations of all vertices in $\widehat{F}(v)$. Then *part-2* is $\sum_{u \in \widehat{F}_j(v)} \Gamma_i^t(u)$. The goal is to prevent not only the source but also the target end-point from being new replica. The sliding window technique in *VP* (see Sec. IV-A) can be used for memory reduction when counting Γ.

[3] $\text{Str}(|N^{\text{out}}|, i)$ is the Stirling set number when partitioning outgoing edges into i nonempty fragments.

Prioritizing Degrees (PD). A vertex as a target might exist in any fragment from any adjacency list. Its counter $\Gamma_i^t(v)$ thereby can be read multiple times. These random reads incur runtime latency. Inspired by the power-law degree distribution, we know high-degree vertices frequently exist in fragments and contribute the majority of replicas. Our improvement is then select the vertex with the maximal degree to count its expectation. For *part-2*, we have the following implementation,

$$\delta\Gamma_i^t(x), x = \arg \max_{u \in \widehat{F}_j(v)} \left\{ N^{\text{in}}(u) + N^{\text{out}}(u) \right\}.$$

Here, we use δ to compensate for unselected expectations. By our tests, $\delta = 2$ is a preferred setting. Compared with AE, now the number of reads decreases from $|\widehat{F}_j(v)|$ down to 1.

Prioritizing Expectations (PE). We state that it is unnecessary to compute an accurate weight factor. Instead, it is enough if the factor can distinguish the scores of all candidate partitions. Our another improvement is then to prioritize expectation counters and select the largest one as the weight. However, a large number of neighbors of a high-degree vertex naturally contribute to a large expectation counter. But it is difficult to avoid creating a new replica of this vertex since it exists in many fragments. We thereby perform quantization on expectations with corresponding degrees, to reduce the probability of selecting high-degree vertices, as shown below,

$$\delta\Gamma_i^t(x), x = \arg \max_{u \in \widehat{F}(v)} \left\{ \frac{\Gamma_i^t(u)}{N^{\text{in}}(u) + N^{\text{out}}(u)} \right\}.$$

VII. Performance Studies

A. Experimental Setup

Benchmark Partitioners. We use suffixes "VP" and "EP" to separate our vertex-based and edge-based partitioners. For *VP*, *SPN* stands for the basic solution enhanced with Neighbors, and *SPNL* is the advanced version with additional topology Locality. For *EP*, since topology locality is the foundation of building fragments, *SPL* is the basic partitioner while *SPLN* is the advanced variant. Below we list competitors.

- *VP*. We run *LDG* [7] and *FENNEL* [8] to show the efficiency advantage of streaming partitioners. For offline partitioners, the classic *METIS* [3] and the recently published *KarMinPar* [12] are tested as the quality benchmark. The SOTA *HeiStream* [19] streams vertices in a buffered manner where data in the buffer are partitioned via offline heuristics. It attempts to strike a balance between quality and efficiency. We also test the shared-memory multiple-threaded partitioners to validate the effectiveness of our parallel technique. They include *MT-METIS* [28], *XtraPuLP* [13], and *KarMinPar*.
- *EP*. We select Degree-Based-Hashing (*DBH*) [9] and Hybrid Edge Partitioning (*HEP*) [17] as two representatives in terms of efficiency and quality. In particular, the most recently released 2-Phase hybrid partitioner with Linear runtime (*2PS-L*) [18], makes a good compromise, which is our main counterpart. Notably recent works exclude parallel comparison because the benefit of parallel processing

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DESCRIPTION OF GRAPH DATASETS (M: MILLION, B: BILLION)

Graphs	|V|	|E|	Size	Type	Domain	Resp.
uk7s	0.1M	3.0M	17MB	Directed	Web	LAW
berk	0.7M	7.6M	58MB	Directed	Web	SNAP
livejl	4.8M	69M	0.5GB	Directed	Social	SNAP
road	24M	5.8M	0.7GB	Undir.	Road	DIMA
wiki	5.7M	130M	1.0GB	Directed	Web	SNAP
eu15	6.7M	172M	1.4GB	Directed	Web	LAW
indo	7.4M	195M	1.5GB	Directed	Web	LAW
orkut	3.1M	224M	1.6GB	Undir.	Social	SNAP
uk20	19M	298M	2.5GB	Directed	Web	LAW
web	118M	1.0B	9.6GB	Directed	Web	LAW
sk20	51M	1.9B	16GB	Directed	Web	LAW
uk7l	109M	3.9B	34GB	Directed	Web	LAW

is marginal due to frequent locking operations [17], [18]. We also follow this design.

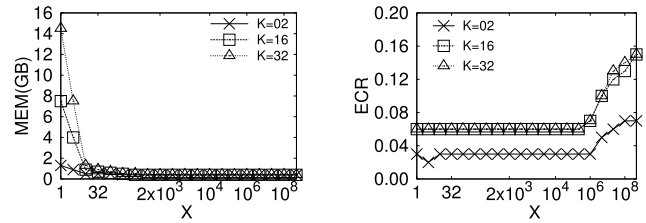
We are aware that there exist two restreaming works for vertex partitioning. They are Lightweight design where high-degree vertices are repeatedly restreamed (marked by *-Light*) [21], and Heavyweight design where vertices are ingested in order of frequently evaluated restreaming benefits (marked by *-Heavy*) [22]. By contrast, our dynamic restreaming heuristic is called as *-Dynamic*. Finally, all partitioners are written in C++ and ingest data in the same order, to make a fair comparison. We implement *LDG* and our proposals by ourself, and use the open-source codes for others.

Platform and Graphs. Our experiments are conducted on a server equipped with dual Intel Xeon Silver 4216 processors (32 cores 2.1GHz), 64GB DDR4 RAM, and hybrid storage consisting of a 4TB 7,200 RPM SATA HDD and a 512GB PCIe 4.0 SSD. We use 12 real graph datasets that have different sizes and stem from different web repositories SNAP[4], LAW[5], and DIMACS[6], as shown in Table I. They have a large diversity about how "easy" we can partition them, which validates the applicability of our proposals. Without loss of generality, all graphs are stored on the HDD, with caching mechanisms active to mitigate storage bottlenecks.

Evaluation Metrics. We focus on both partitioning quality and efficiency. The quality is evaluated by the Edge Cut Ratio (*ECR*) for *VP* and the Replication Factor (*RF*) for *EP*[7]. *ECR* is the ratio of the total number of cutting edges to $|E|$. While *RF* is the average number of replicas of a vertex. On the other hand, the efficiency is measured by partitioning *Runtime* starting from the point loading the first data to the point saving the assignment route table. Each test case is executed five times, and we report the best observed runtime performance. The Memory consumption (*MEM*) is also tested to validate the effectiveness of our shard technique.

Below, we organize experiments around our three proposals, the enhanced heuristics for *VP* (Sec. VII-B), followed by the restreaming optimization (Sec. VII-C), and the applicability on *EP* (Sec. VII-D). Besides, we explore the impact of different storage media (Sec. VII-E) and the capability of seeking a sweet spot between efficiency and quality (Sec. VII-F). The symbol 'F' indicates an unsuccessful run due to the "Out Of Memory" (OOM) error. We set $K = 32$ by default.

[4]https://snap.stanford.edu/data/
[5]https://law.di.unimi.it/datasets.php
[6]http://www.diag.uniroma1.it/challenge9/download.shtml
[7]Workload balance is another quality metric but we ignore it for brevity, because all partitioners have the similar performance by our tests.



(a) *MEM* (b) *ECR*

Fig. 8. Impact of the shard number X (*SPNL-VP*, *web*).

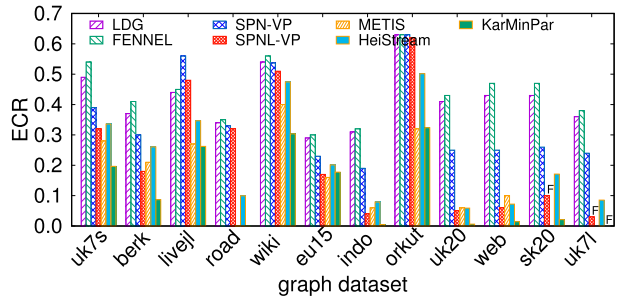


Fig. 9. Quality comparison for vertex partitioning.

B. Evaluation on Vertex Partitioning

This group of experiments first explore a prominent shard granularity for memory reduction (Sec. IV-A), and then compare our proposals against existing works.

Memory Reduction. We run *SPNL-VP* over *web* as a case to explore the impact of the shard number X. Fig. 8(a) shows that *MEM* significantly decreases when X increases. Given an extremely large X, the over-small shard does not dominate the total memory consumption any more, but yields a clear degradation of *ECR*, as shown in Fig. 8(b). In general, a quite large range of X can reduce *MEM* without negative impact on *ECR*. Here we empirically infer $X = \min\{\alpha K, \frac{|V|}{\beta K}\}$ with $\alpha = 4$ and $\beta = 10^4$ by default. Taking *web* with $K = 32$ as an example, we have $X = 128$. By this setting, the *MEM* of *SPNL-Vert* drops from 14.5GB ($X = 1$) down to 0.6GB, which is competitive to *LDG* (0.4GB).

Partitioning Comparison. As shown in Figs. 9 and 10, the online *LDG*/*FENNEL* and offline *KarMinPar* have different favorites in the efficiency and quality metrics. *KarMinPar* generally outperforms *METIS* except the *ECR* comparison on *eu15*. However, *KarMinPar* still consumes many memory resources, and even encounters OOM on *uk7l*. By contrast, *HeiStream* makes a compromise. Our proposals follow this research line but go further. They consistently run faster than *HeiStream*

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

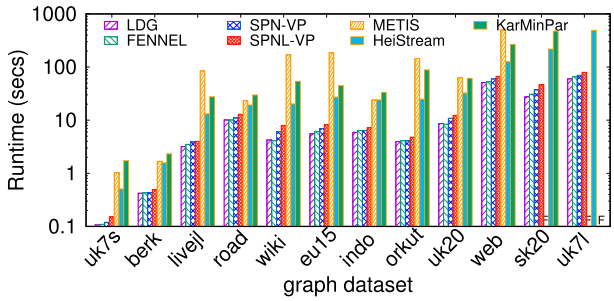


Fig. 10. Efficiency comparison for vertex partitioning.

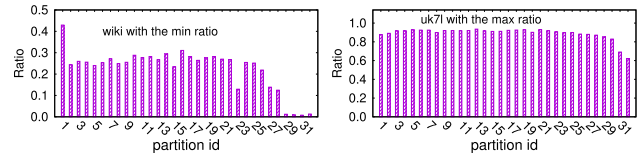
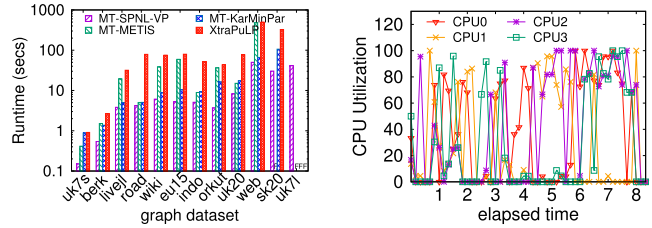


Fig. 11. The capability of capturing topology locality by pre-assignment.



(a) Overall runtime (b) CPU utilization (*uk20*)

Fig. 12. Parallel vertex partitioning.

while simultaneously achieving a comparable and even better *ECR* reduction. In particular, *SPN-VP* reduces *ECR* by at most 42% compared to *LDG* (*web*, 16% on average); and runs 20X faster at most than *KarMinPar* (*orkut*, 7X on average). Adding topology locality creates another gap between *SPN* and *SPNL* with marginal runtime latency. The *ECR* improvement increases to 92% (*uk7l*, 47% on average). The runtime speedup against *KarMinPar* is 17X (6X on average). Even compared with *HeiStream*, the speedup of *SPNL-VP* is still up to 4X (*orkut* and *uk7s*, 2.4X on average), and the *ECR* improvement is 65% at most (*uk7l*).

We observe significant variation in the *ECR* performance gap between *SPN* and *SPNL* across different graphs. To investigate this phenomenon, we evaluate the effectiveness of our Range-based pre-assignment in capturing topology locality, measured by the fraction of vertices that retain their pre-assigned partitions after executing *SPNL*. A high ratio indicates strong locality-capturing capability, which translates to substantial *ECR* reduction. Our experiments reveal two distinct categories of graphs. High-ratio graphs (*berk*, *indo*, *uk20*, *web*, *sk20*, and *uk7l*) exhibit ratios between 0.66 and 0.89, leading to significant *ECR* improvements (Fig. 9). Low-ratio graphs show weak locality capture (ratios of 0.23–0.55), resulting in limited *ECR* optimization. Fig. 11 illustrates this contrast by comparing detailed ratios of all partitions on two representative graphs *wiki* and *uk7l*, respectively exhibiting the lowest and the highest average ratios. Notably, among underperforming graphs, undirected *road* and *orkut* graphs inherently include both in- and out-neighbors in adjacency lists. Additionally, small-world social networks (*livejl*, *orkut*) and large-diameter graphs (*road*, *wiki*) exhibit highly dispersed in-neighbor distributions across adjacency lists. These factors—complete in-out neighborhood information and structural dispersion—hinder the learning of implicit knowledge from local views. Consequently, when locality capture is poor, *SPNL* degrades to *LDG*. However, restreaming can alleviate this degradation by leveraging partition results from previous passes to compensate for missing knowledge embedded in local in-neighbor view and global pre-assignment view.

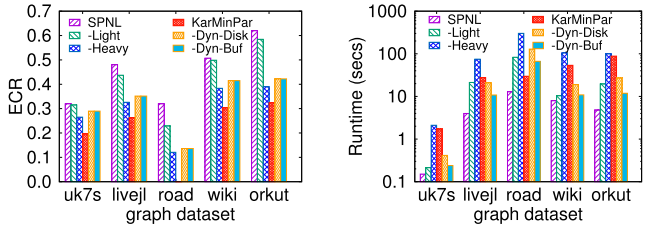
Parallel Optimization. For *METIS*, *KarMinPar*, and our proposed *SPNL-VP*, we additionally investigate the runtime performance of their shared-memory Multi-Threaded (MT) implementations (denoted by the "MT" prefix). Our evaluation also includes the MT version of *XtraPuLP*. For each graph dataset, we conduct multiple runs with varying degrees of parallelism and select the configuration yielding optimal runtime performance. As shown in Fig. 12(a), *MT-SPNL-VP* achieves significant speedups: up to 9X faster than *MT-METIS*, 14X faster than *XtraPuLP*, and 5X faster than *MT-KarMinPar*. It is generally comparable in runtime to the fast *LDG* approach, and even achieves 31% reduction on *uk7l*. We observe that all parallel partitioning approaches introduce random perturbations in assignment decisions. *XtraPuLP* exhibits an average 15% degradation in ECR compared against its single-thread version. In contrast, *METIS* and *KarMinPar* mitigate this impact through multi-level refinement, with degradation below 5%. *SPNL* reduces this effect to under 3% by decoupling dependency among vertices.

For our *MT-SPNL* we examine the negative impact of cross-thread synchronization on *uk02*. As we scale the thread count from 1 to 64, the partitioning runtime initially decreases from 12.3s to 8.2s (optimal at 4 threads), but subsequently increases to 15s due to contention overhead. Fig. 12(b) further demonstrates the computing power utilization across the four active CPU cores. The average core utilization remains at approximately 35.2%. This underutilization is due to I/O latency caused by loading the input graph and cross-thread synchronization stalling from atomic operations. In Sec. VII-E, we find that the utilization increases to 51.6% when graphs are resident in memory (eliminating the I/O bottleneck).

C. Evaluation on Restreaming Vertex Partitioning

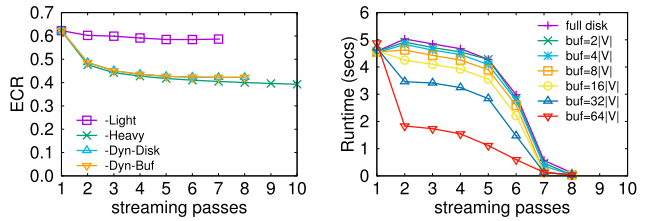
We adapt three restreaming models (*-Light*, *-Heavy*, and *-Dynamic*) to our streaming *SPNL-VP* heuristic. The upper bound of restreaming passes is set as 10, but the iteration can be terminated in advance if *ECR* keeps unchanged. For *-Dynamic*, the iterative procedure gradually converges by dynamically tuning its restreaming threshold. Note that previous experiments have confirmed the prominent performance of *SPNL-VP* on a

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.



(a) Quality comparison (b) Efficiency comparison

Fig. 13. Effectiveness of restreaming vertex partitioning.



(a) Quality variation (b) Efficiency variation of *-Dynamic*

Fig. 14. Detailed behavior analysis during restreaming passes (*orkut*).

few graphs. The optimization space left for restreaming is very limited. We exclude them for brevity.

Fig. 13 reports *ECR* and accumulated runtime after the restreaming iteration terminates. Here *Disk* and *Buf* are two branches of *Dynamic* w/o and w/ the caching design (Sec. V-C). Compared with *SPNL-VP*, we observe that *-Heavy* achieves the best *ECR* improvement (up to 62%) but incurs the heaviest *Runtime* latency (22X at most). *Light* pursues another extreme where the maximal factors are 28% and 5X on the two metrics. By contrast, our *-Dyn-Disk* optimizes *ECR* by 57% at most with roughly 9X latency. *-Dyn-Buff* maintains the quality improvement but further reduces the latency to 4X, which is even faster than *Light*[8]. For a comprehensive comparison, we also include *KarMinPar* in our evaluation. Note that in real-world graph systems, restreaming progressively improves *ECR* through reloading operations of multiple jobs with the same input. Consequently, the accumulated runtime reported in Fig. 13(b) should be amortized[9] and then compared against that of *KarMinPar*. Clearly, *KarMinPar* achieves the highest *ECR* reduction at the expense of significantly large runtime penalty. Furthermore, as demonstrated in Fig. 9 (particularly for "F" on *uk7l*), *KarMinPar*'s substantial memory requirements present an additional scalability challenge.

For better understanding how our technique works, Fig. 14 gives detailed behavior analysis during passes, using *orkut* as an example. Fig. 14(a) reveals that the large *ECR* reduction is always achieved in the early phase. Our model can catch the wind and then adaptively adjust the restreaming selection threshold θ, so as to avoid restreaming less significant vertices. As a result, Fig. 14(b) shows the gradual decrease of runtime. It also explores the impact of our caching design by varying the buffer size from zero (*full disk*) to $64|V|$ (nearly caching the whole graph). In particular, after the 6-th passes, a majority of vertices are excluded from restreaming due to the small θ. In other words, nearly all of restreamed vertices can be cached, even with a small buffer size. The runtime gap of different buffer sizes thereby quickly narrows.

D. Evaluation on Edge Partitioning

We investigate the overall performance of edge partitioners. Among them, *HEP* divides edges into two parts and then partitions them respectively with offline and online heuristics. We

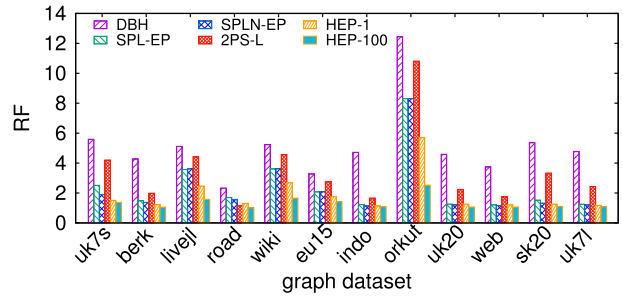


Fig. 15. Quality comparison for edge partitioning.

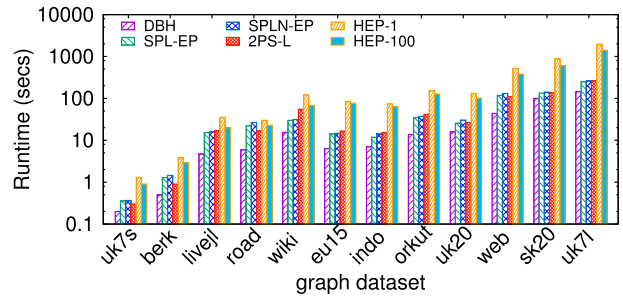


Fig. 16. Efficiency comparison for edge partitioning.

test it using two division thresholds 1 and 100 as preferred in the original paper [17]. The latter enforces more edges to be partitioned in offline and hence enjoys a lower *RF* value, but at expense of memory consumption [18]. *DBH* quickly partitions edges based on simple degree information.

As presented in Figs. 15 and 16, *DBH* and *HEP* clearly have different advantages in terms of efficiency and quality. While, *2PS-L* and our proposals strike a good balance between the two metrics. Benefiting from the topology locality enhancement, *RF* of our basic *SPL-EP* is lower than that of *2PS-L* (up to 54% on *sk20* and 26% on average). *SPLN-EP* with the best policy of utilizing expectations further reduces *RF* by 24% at most (*uk7s*). The *RF* reduction compared against *2PS-L* then increases to 60% at most and 30% on average. One exception is *road* where its low average degree heavily limits the benefit of sharing the common source vertex in a fragment. While for the runtime metric, both of our two proposals have the performance comparable to *2PS-L*, and even achieve roughly 40% reduction on *wiki*.

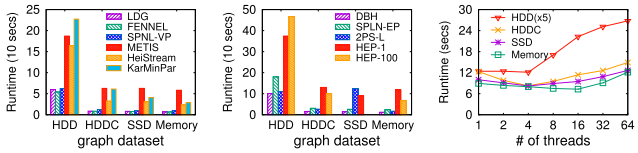
[8]These maximal improvement and latency are both achieved on road.

[9]It is the runtime of a single restreaming iteration, which is comparable to that of *SPNL-VP*.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF THREE POLICIES WHEN UTILIZING REPLICATION EXPECTATIONS

Graphs	*RF*			*Runtime* (secs)		
	AE	PD	PE	AE	PD	PE
uk7s	2.59	2.67	1.89	0.5	0.5	0.4
berk	1.52	1.53	1.37	1.7	1.5	1.5
livejl	3.57	3.58	3.61	18.5	19.3	15.9
road	1.69	1.69	1.56	28.0	27.0	26.5
wiki	3.75	3.72	3.62	34.0	37.2	31.2
eu15	2.08	2.09	2.08	24.2	19.5	14.6
indo	1.25	1.26	1.14	19.6	16.6	14.1
orkut	8.39	8.45	8.31	45.0	43.7	37.1
uk20	1.27	1.28	1.20	36.6	32.0	30.4
web	1.21	1.22	1.16	163.8	153.0	130.8
sk20	1.58	1.63	1.32	195.4	172.9	141.2
uk7l	1.31	1.31	1.20	366.6	317.7	260.4



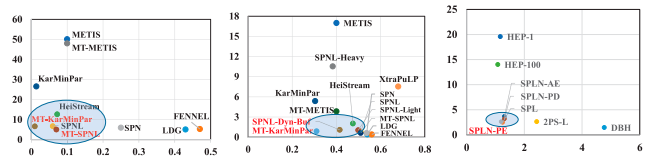
(a) Vertex partitioning (b) Edge partitioning (c) Parallel optimization

Fig. 17. Runtime comparison on different storage media (*uk20*).

We next explore the difference when utilizing replication expectations in different policies, i.e., *AE*, *PD* and *PE* proposed in Sec. VI-C. Table II summarizes the results. *PD* generates fewer random reads than *AE*. It thereby accelerates partitioning by up to 19% (*eu15*), with negligible degradation of *RF* (less than 3%). *PE* beats *PD* in most cases. We guess the reason is that the sum of expectations contributed by all vertices might confuse the heuristic. The partitioner cannot distinguish candidate partitions very well. Another observation is that under *PE*, the selected representative is usually the source vertex. We then compare the source with other sampled vertices to roughly perform the priority selection. Such a variant runs 40% faster than *AE*, at most. Even compared with *PD*, the improvement is still up to 25%. We thereby use the sampling-based *PE* as the default policy.

E. Evaluation on the Impact of Storage Media

Finally, we investigate the impact of various storage media on runtime performance. We simulate four distinct environments where graphs are stored on HDD without caching (HDD), HDD with active caching mechanisms (HDDC), SSD, and in-Memory storage. Fig. 17 presents the runtime comparison for both vertex and edge partitioning approaches. It shows substantial performance degradation for all partitioning methods (up to 6X and 4X on average) when using the conventional bandwidth-limited HDD device. In contrast, although performance differences exist between HDDC, SSD, and in-memory storage, the gap is relatively small. This is because graph partitioning is a hybrid workload with I/O and computational demands. Now the primary bottleneck shifts to computing assignment scores rather than data access.



(a) VP on *web* (b) VP on *wiki* (c) EP on *uk7l*

Fig. 18. Scatter plots of runtime ($\times 10$ secs for VP and $\times 10^2$ secs for EP) versus quality (*ECR* for VP and *RF* for EP).

We then further evaluate the impact on parallel computation optimization. As shown in Fig. 17(c), on HDDs, significant I/O latency forces CPUs into prolonged idle states, resulting in negligible runtime improvements (<2%). In contrast, HDDC and SSD alleviate this bottleneck and deliver consistent performance gains (33% and 18% respectively). Notably, for memory-resident graphs that eliminate disk I/O entirely, we observe a significant increase in average CPU utilization from 35.2% (with HDDC/SSD) to 51.6%. This enhanced utilization allows the system to accommodate additional synchronization overheads, thereby enabling high degrees of parallelism (16 threads versus the 4-thread limit observed with HDD/HDDC/SSD configurations).

F. Evaluation on Balancing Efficiency and Quality

Fig. 18 presents a runtime comparison against *ECR*/*RF* ratios, illustrating the trade-off between efficiency and partitioning quality across all evaluated approaches. For vertex partitioning (VP), we analyze the largest graphs that all methods can work successfully. The multi-threaded offline approach *KarMinPar* and our online *SPNL* demonstrates comparable performance to each other in terms of quality-efficiency balance. However, *KarMinPar* exhibits significant limitations in space scalability, failing to process the *uk7l* dataset due to OOM errors. Notably, our approach maintains competitive performance even on the challenging *wiki* graph, which contains deeply embedded knowledge in both local and global views. In edge partitioning (EP) scenarios, our *SPLN-PE* method significantly outperforms competing approaches, achieving the optimal balance between computational efficiency and partition quality.

Notably, there exist many implementations to extract implicit knowledge from both local and global views, including constructing in-neighbors during a preprocessing phase or in the first restreaming iteration, and initiating partitioning from outgoing edges rather than employing simple range pre-assignment. However, the former introduces additional I/O overhead when loading in-neighbor data (yielding an average runtime increase of 24.3% even on SSD), while the latter necessitates processing the graph twice, doubling the total runtime. Both approaches consequently compromise the runtime efficient advantage of the lightweight streaming partitioning.

Recall that the execution pipeline of our *SPNL* consists of loading input graph, our novel in-neighbor analysis, the newly proposed pre-assigned data analysis, and conventional out-neighbor analysis. We have quantified the runtime distribution for each component across diverse graph datasets. The average

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

time distribution (measured on SSDs) is 24.3%, 12.5%, 14.2%, and 49.1%, respectively, indicating that our two knowledge extraction principles incur lightweight runtime penalty.

VIII. Related Works

We begin with an introduction to offline and streaming branches, each of which is further divided into vertex- and edge-based, and then overview other complementary works.

A. Offline Graph Partitioning

Vertex-based: Early attempts include bisection algorithms like Kernighan-Lin and its variant [29], and the representative k-way multilevel algorithm *METIS* [3]. *METIS* partitions the coarsest graph from the input and then projects back the initial division for refinement. Its design has been widely adopted. For example, Schweitz et al. [30] develop a partitioner that uses domain knowledge to decompose sequential programs into parallel tasks. However, these multilevel partitioners require to repeatedly access graph data, leading to expensive runtime costs. For efficiency, many parallel variants typically like the shared-memory *MT-METIS* [28] and its variant [11] have been proposed, but the improvement is limited as validated in our tests. Another alternative is to use fast label propagation to replace components in *METIS* [31], that facilitates parallel partitioning [32]. Slota et al. extend this idea in their *XtraPuLP* partitioner with multiple constraints and multiple objectives [13]. Recently, Lars et al. partition the coarsest graph by jointly running recursive bipartitioning and direct k-way partitioning [12]. The proposed *KarMinPar* has prominent quality and efficiency.

Edge-based: Bourse et al. propose the earliest offline edge partitioning by extending vertex-based *METIS* as a subroutine [5]. Although this partitioner achieves the state-of-the-art replication factor, the poor scalability motivates many subsequent studies. They simplify heuristic computations by sacrificing certain quality. The detailed techniques include constructing core vertex sets [6], elimination trees [14], and split graphs [33], and parallel implementation [14], [15].

B. Streaming/Online Graph Partitioning

Vertex-based: *LDG* [7] and *FENNEL* [8] are the earliest and the most popular approaches in literature. Faraj et al. buffer vertices so as to run inefficient multilevel policies for high quality [19]. As validated, restreaming the entire or partial output of a previous execution can also improve quality [20], [21], [22], but the runtime penalty is large. There also exists shared-memory parallel computation for efficiency [34], but generates large quality degradation as reported.

Edge-based: Early pioneers such as Oblivious and Greedy keep track of the historic assignments to decide the placement of the current edge [2]. DBH quickly makes the decision by degree-based hashing values [9]. HDRF combines these heuristics for better quality [10]. Notably edges that encounter the assignment dilemma can be temporarily buffered until sufficient knowledge is available [35]. Expensive re-assignments and/or sophisticated heuristics can also be run for these buffered edges [19], [36], [37]. Instead of passively buffering edges, researchers also actively combine different heuristics. OffStream runs offline partitioner [6] and streaming HDRF for different edges [16]. HEP further optimizes the time-space complexity of NE [17]. The recent 2PS-L even replaces NE with a lightweight clustering [18].

C. Other Complementary Optimizations

We also investigate many important but complementary works. First, several studies explore the impact of different streaming orderings on partitioning quality [7], [22]. The simple and effective Degree ordering is usually preferred, especially for restreaming [21], [22]. Our restreaming principle also integrates this design. Second, the partitioner serves as the foundation for advanced techniques where our proposals can work as a plugged component. Hybrid approaches like PowerLyra [23], IOGP [24], and GraPU [25] improve quality by partitioning both vertices and edges. Meanwhile, other systems build upon partitioned results for further optimizations. For instance, Zhang et al. [38] use edge-based partitioning to enable fast asynchronous message propagation, while Li et al. [39] optimize communication patterns rather than just message volume. Similarly, vertex-based *METIS* partitioning facilitates parallel Graph Neural Network (GNN) training [40]. Finally, dynamic graph partitioning attracts much attention [25], [41], [42]. Although this is orthogonal to our focus, our partitioner, however, can cope with the inserting-only changes by regarding inserted vertices/edges as newly arrived data.

IX. Conclusion

This paper investigates the shortcomings of existing streaming and offline partitioners, and then proposes a series of new heuristics to fully utilize knowledge for both vertex- and edge-based streaming partitioning. Optimizations about memory consumption, parallel acceleration and restreaming design are also proposed to further enhance the performance. Experiments validate that the proposed solutions achieve an overall success in quality and efficiency.

References

[1] G. Malewicz et al., "Pregel: A system for large-scale graph processing," in *Proc. SIGMOD*, New York, NY, USA: ACM, 2010, pp. 135–146.
[2] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin, "PowerGraph: Distributed graph-parallel computation on natural graphs," in *Proc. OSDI. USENIX Assoc.*, 2012, pp. 17–30.
[3] G. Karypis and V. Kumar, "Multilevel k-way partitioning scheme for irregular graphs," *J. Parallel Distrib. Comput.*, vol. 48, no. 1, pp. 96–129, 1998.
[4] Ü. V. Çatalyürek et al., "More recent advances in (hyper) graph partitioning," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
[5] F. Bourse, M. Lelarge, and M. Vojnovic, "Balanced graph edge partition," in *Proc. SIGKDD*, New York, NY, USA: ACM, 2014, pp. 1456–1465.
[6] C. Zhang, F. Wei, Q. Liu, Z. G. Tang, and Z. Li, "Graph edge partitioning via neighborhood heuristic," in *Proc. SIGKDD*, New York, NY, USA: ACM, 2017, pp. 605–614.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

[7] I. Stanton and G. Kliot, "Streaming graph partitioning for large distributed graphs," in *Proc. SIGKDD*, New York, NY, USA: ACM, 2012, pp. 1222–1230.

[8] C. E. Tsourakakis, C. Gkantsidis, B. Radunovic, and M. Vojnovic, "FENNEL: Streaming graph partitioning for massive scale graphs," in *Proc. WSDM*, New York, NY, USA:ACM, 2014, pp. 333–342.

[9] C. Xie, L. Yan, W. Li, and Z. Zhang, "Distributed power-law graph computing: Theoretical and empirical analysis," in *Proc. NeurIPS*, 2014, pp. 1673–1681.

[10] F. Petroni, L. Querzoni, K. Daudjee, S. Kamali, and G. Iacoboni, "HDRF: Stream-based partitioning for power-law graphs," in *Proc. CIKM*, New York, NY, USA: ACM, 2015, pp. 243–252.

[11] Y. Akhremtsev, P. Sanders, and C. Schulz, "High-quality shared-memory graph partitioning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 11, pp. 2710–2722, Nov. 2020.

[12] L. Gottesbüren, T. Heuer, P. Sanders, C. Schulz, and D. Seemaier, "Deep multilevel graph partitioning," in *Proc. 29th Annu. Eur. Symp. Algorithms (ESA)*, 2021, vol. 204, pp. 48:1–48:17.

[13] G. M. Slota, C. Root, K. D. Devine, K. Madduri, and S. Rajamanickam, "Scalable, multi-constraint, complex-objective graph partitioning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 12, pp. 2789–2801, Dec. 2020.

[14] D. W. Margo and M. I. Seltzer, "A scalable distributed graph partitioner," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1478–1489, 2015.

[15] M. Hanai, T. Suzumura, W. J. Tan, E. S. Liu, G. Theodoropoulos, and W. Cai, "Distributed edge partitioning for trillion-edge graphs," *Proc. VLDB Endowment*, vol. 12, no. 13, pp. 2379–2392, 2019.

[16] T. Ayall, H. Duan, C. Liu, F. Gereme, M. Abegaz, and M. Deleli, "Taking heuristic based graph edge partitioning one step ahead via offstream partitioning approach," in *Proc. ICDE*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 2081–2086.

[17] R. Mayer and H. Jacobsen, "Hybrid edge partitioner: Partitioning large power-law graphs under memory constraints," in *Proc. SIGMOD*, New York, NY, USA: ACM, 2021, pp. 1289–1302.

[18] R. Mayer, K. Orujzade, and H. Jacobsen, "Out-of-core edge partitioning at linear run-time," in *Proc. ICDE*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 2629–2642.

[19] M. F. Faraj and C. Schulz, "Buffered streaming graph partitioning," *ACM J. Exp. Algorithmics*, vol. 27, no. 1, pp. 1–26, 2022.

[20] J. Nishimura and J. Ugander, "Restreaming graph partitioning: Simple versatile algorithms for advanced balancing," in *Proc. SIGKDD*, 2013, pp. 1106–1114.

[21] G. Echbarthi and H. Kheddouci, "Fractional greedy and partial restreaming partitioning: New methods for massive graph partitioning," in *Proc. IEEE Int. Conf. BigData*, Washington, DC, USA: IEEE Computer Society, 2014, pp. 25–32.

[22] A. Awadelkarim and J. Ugander, "Prioritized restreaming algorithms for balanced graph partitioning," in *Proc. SIGKDD*, New York, NY, USA: ACM, 2020, pp. 1877–1887.

[23] R. Chen, J. Shi, Y. Chen, B. Zang, H. Guan, and H. Chen, "PowerLyra: Differentiated graph computation and partitioning on skewed graphs," *ACM Trans. Parallel Comput.*, vol. 5, no. 3, pp. 13:1–13:39, 2018.

[24] D. Dai, W. Zhang, and Y. Chen, "IOGP: An incremental online graph partitioning algorithm for distributed graph databases," in *Proc. HPDC*, New York, NY, USA: ACM, 2017, pp. 219–230.

[25] F. Sheng, Q. Cao, and J. Yao, "Exploiting buffered updates for fast streaming graph analysis," *IEEE Trans. Comput.*, vol. 70, no. 2, pp. 255–269, Feb. 2021.

[26] Z. Wang et al., "Lightweight streaming graph partitioning by fully utilizing knowledge from local view," in *Proc. IEEE 43rd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 614–625.

[27] N. Wang, Z. Wang, Y. Gu, Y. Bao, and G. Yu, "TSH: Easy-to-be distributed partitioning for large-scale graphs," *Future Gener. Comput. Syst.*, vol. 101, pp. 804–818, Dec. 2019.

[28] D. Lasalle and G. Karypis, "Multi-threaded graph partitioning," in *Proc. 27th Int. Symp. on Parallel Distrib. Process.*, Washington, DC, USA: IEEE Computer Society, 2013, pp. 225–236.

[29] P. Chardaire, M. Barake, and G. P. McKeown, "A PROBE-based heuristic for graph partitioning," *IEEE Trans. Comput.*, vol. 56, no. 12, pp. 1707–1720, Dec. 2007.

[30] E. A. Schweitz and D. P. Agrawal, "A parallelization domain oriented multilevel graph partitioner," *IEEE Trans. Comput.*, vol. 51, no. 12, pp. 1435–1441, Dec. 2002.

[31] P. Sanders and C. Schulz, "Engineering multilevel graph partitioning algorithms," in *Proc. 19th Annu. Eur. Symp. Algorithms (ESA)*, New York, NY, USA: Springer-Verlag, 2011, vol. 6942, pp. 469–480.

[32] J. Ugander and L. Backstrom, "Balanced label propagation for partitioning massive graphs," in *Proc. WSDM*, New York, NY, USA: ACM, 2013, pp. 507–516.

[33] S. Schlag, C. Schulz, D. Seemaier, and D. Strash, "Scalable edge partitioning," in *Proc. 21st Workshop Algorithm Eng. Experiments,* Philadelphia, PA, USA: SIAM, 2019, pp. 211–225.

[34] M. F. Faraj and C. Schulz, "Recursive multi-section fly: Shared-memory streaming algorithms hierarchical graph partitioning process mapping," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 473–483.

[35] Y. Li, C. Li, A. Orgerie, and P. R. Parvédy, "WSGP: A window-based streaming graph partitioning approach," in *Proc. IEEE/ACM 21st Int. Symp. Cluster, Cloud Internet Comput. (CCGrid)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 586–595.

[36] C. Mayer et al., "ADWISE: Adaptive window-based streaming edge partitioning for high-speed graph processing," in *Proc. IEEE 38th Int. Conf. Distrib. Comput. Syst. (ICDCS),* Piscataway, NJ, USA: IEEE Press, 2018, pp. 685–695.

[37] Q. Hua, Y. Li, D. Yu, and H. Jin, "Quasi-streaming graph partitioning: A game theoretical approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 7, pp. 1643–1656, Jul. 2019.

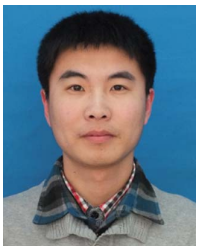
[38] Y. Zhang, X. Liao, H. Jin, L. Gu, G. Tan, and B. B. Zhou, "HotGraph: Efficient asynchronous processing for real-world graphs," *IEEE Trans. Comput.*, vol. 66, no. 5, pp. 799–809, May 2017.

[39] Z. Li, X. Chen, Y. Yang, F. Min, X. Zhang, and Y. Han, "A data-centric software-hardware co-designed architecture for large-scale graph processing," *IEEE Trans. Comput.*, vol. 74, no. 4, pp. 1109–1122, Apr. 2025.

[40] Z. Xue, Y. Yang, and R. Marculescu, "SUGAR: Efficient subgraph-level training via resource-aware graph partitioning," *IEEE Trans. Comput.*, vol. 72, no. 11, pp. 3167–3177, Nov. 2023.

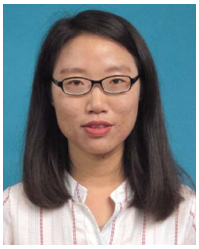
[41] M. A. K. Patwary, S. Garg, S. K. Battula, and B. Kang, "SDP: Scalable real-time dynamic graph partitioner," *IEEE Trans. Services Comput.*, vol. 16, no. 1, pp. 564–574, Jan./Feb. 2023.

[42] A. McCrabb and V. Bertacco, "Optimizing vertex pressure dynamic graph partitioning in many-core systems," *IEEE Trans. Comput.*, vol. 70, no. 6, pp. 936–949, Jun. 2021.



Zhigang Wang (Member, IEEE) received the Ph.D. degree in computer software and theory from the Northeastern University, China, in 2018. He is currently an Associate Professor with Guangzhou University. His research interests include distributed graph processing and machine learning. He is a Member of IEEE and the China Computer Federation (CCF). He won CCF Outstanding Doctoral Dissertation Award (2018), ACM QINGDAO Rising Star Award (2019), and the second prize of CCF Natural Science Award (2023).

Gongtai Sun received the bachelor's degree from Qingdao University, China, in 2023. He is currently working toward the master's degree in the School of Computer Science and Technology with the Ocean University of China. His research interest is large-scale graph analysis.



Ning Wang received the Ph.D. degree in computer software and theory from the Northeastern University, China, in 2017. She is currently an Associate Professor with Guangzhou University. She has been a Visiting Ph.D. Student in Nanyang Technological University during 2014 to 2015. Her research interests include big data process, graph analysis, and differential privacy protection. She won ACM QINGDAO Rising Star Award (2021)

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.

Lixin Gao (Fellow, IEEE) received the Ph.D. degree in computer science from UMASS Amherst, in 1996. She is currently a Distinguished Professor with UMASS Amherst. Her research interests include social networks and cloud computing. She was an Alfred P. Sloan Fellow and received an NSF CAREER Award. She won the Best Paper Award from IEEE INFOCOM and ACM SoCC, and the Test-of-Time Award in ACM SIGMETRICS. She received the Chancellors Award for Outstanding Accomplishment in Research and Creative Activity, and is a Fellow of IEEE and ACM.

Chuanfei Xu received the B.E. degree in 2007 from Shenyang University of Technology, the M.E. degree and Ph.D. degree in 2009 and 2013 from computer science, Northeastern University, China. He is currently an Associate Researcher in the Central Research Institute, Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ). His research interests include spatial database management, parallel computing and natural language processing (NLP).

Yu Gu received the Ph.D. degree in computer software and theory from the Northeastern University, China, in 2010. He is currently a Professor and the Ph.D. Supervisor with the Northeastern University, China. His research interests include big data analysis, distributed computation and graph data management. He is a Senior Member of CCF. He won the second prize of CCF Natural Science Award in 2023.

Ge Yu (Senior Member, IEEE) received the Ph.D. degree in computer science from Kyushu University of Japan, in 1996. He is currently a Professor and the Ph.D. supervisor with the Northeastern University of China. His research interests include distributed and parallel database, OLAP and data warehousing, data integration, graph data management, etc. He is a Senior Member of IEEE Computer Society, IEEE, ACM, and a Fellow of CCF. He won the second prize of CCF Natural Science Award in 2023.

Zhihong Tian (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from Harbin Institute of Technology. He is currently a Professor and Dean with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong, China. He is also a part-time Professor with Carlton University. He has authored more than 200 journal and conference papers in computer networks and cyberspace security. He is a Senior Member of the IEEE Computer Society and CCF.

Authorized licensed use limited to: GUANGZHOU UNIVERSITY. Downloaded on February 22,2026 at 09:20:59 UTC from IEEE Xplore. Restrictions apply.